

ET4351 Digital VLSI SoC

Lab Report

(Feedback Only, Not Graded)

Student Name: Daniel Tyukov
Student Number: 5714699

Due Date: March 6, 2026

Lab Report

1 Lab 1: Project Baseline with PicoSoC

Question 1.1: For Task 1, provide a detailed, step-by-step explanation of the testbench's operation.

(a) How does PicoSoC load the firmware? (Hint: Check `spiflash.v`)

Answer 1.1(a)

The testbench (`et4351_tb.sv`) instantiates the `et4351` top-level module (the DUT) and a spi-flash module. During initialization, `spiflash.v` loads the firmware hex file into its internal memory array using `$readmemh(firmware_file, memory)`, where the firmware path is passed via the `+firmware=plusarg`. At reset, the PicoRV32 CPU inside PicoSoC begins fetching instructions starting from `PROGADDR_RESET = 0x0010_0000` (1 MB into the flash address space). The `spimemio` module in `picosoc.v` handles SPI flash reads: when the CPU issues a memory access in the flash range (address $\geq 4 \cdot \text{MEM_WORDS}$ and $< 0x0200_0000$), `spimemio` drives the SPI protocol signals (`csb`, `clk`, `io0-io3`) to the external `spiflash` model, which responds with the requested data bytes. The flash model supports multiple SPI modes (standard, dual, quad, DDR) for increasing throughput.

(b) How does PicoSoC display the “Hello, World!” message on QuestaSim’s command line? Explain to which memory address the ASCII characters are written by the `print_str` function in `hello.c` and how PicoSoC sends out the characters to the testbench through the `ser_tx` port. (Hint: Check `simpleuart.v`, `picosoc.v`, `uart.c`, `uart.h` and other related files.)

Answer 1.1(b)

The C firmware (`hello.c`) calls `init_uart()` which sets the UART clock divider register at address `0x02000004` to 104, configuring the baud rate. Then `print_str("Hello, World!\n")` iterates over each character and calls `print_char(c)`, which writes the character to the UART data register at address `0x02000008` (`reg_uart_data`). In `picosoc.v`, writes to `0x02000008` are routed to `simpleuart's reg_dat_we` input. The `simpleuart` module serializes each byte into a UART frame (start bit, 8 data bits, stop bit) and shifts it out on `ser_tx` at the configured baud rate. The testbench (`et4351_tb.sv`) monitors `ser_tx` with an always block that detects the start bit (negedge), samples 8 data bits, and uses `$write` to display each character on the QuestaSim console. When `print_char(-1)` sends a value > 127 , the testbench terminates the simulation via `$finish` after printing the cycle count (26,554 cycles).

Question 1.2: For Task 2, carefully examine the Verilog code for the dummy accelerator and explain its functionality. How is the accelerator launched? How does it send its output to the RISC-V core with a handshake for robust data exchange?

Answer 1.2

The dummy accelerator (`accelerator.v`) is a counter-based module mapped to address range `0x0300.0000`. It has 4 memory-mapped registers: GPIO (`0x03000000`), CSR (`0x03000004`), Input (`0x03000008`), and Output (`0x0300000C`). The accelerator is **launched** by the CPU writing to the CSR register: bit 0 controls reset (active-high), bit 1 enables the accelerator, and bit 2 is a read-only done flag. The memory interface responds when `iomem_valid` is asserted and `iomem_addr[31:24] == 0x03`. Reads and writes to the 4 registers are handled byte-by-byte using `iomem_wstrb`.

The **handshake** mechanism works as follows: the CPU writes the target count to the Input register, then sets the enable bit in CSR. The accelerator increments an internal counter each clock cycle until it reaches the target value. Upon completion, it copies the counter value to the result register and sets the `valid_out` flag, which is reflected in CSR bit 2. The CPU polls CSR bit 2 and reads the Output register when done. This is a simple polling-based handshake: the CPU starts the accelerator via CSR and waits for the done flag.

Question 1.3: In Task 2, closely inspect the `dummy_accel.c` file and provide a thorough explanation (by annotating the relevant C and Verilog code) of how the PicoRV32 core interacts with the dummy accelerator, such as sending inputs to and receiving outputs from the accelerator.

Answer 1.3

In `dummy_accel.c`, the PicoRV32 core interacts with the accelerator through memory-mapped I/O registers defined as volatile pointers. The interaction follows these steps:

1. **Initialize:** All four registers (GPIO, CSR, Input, Output) are written to `0x00000000`. This is a write to addresses `0x03000000–0x0300000C` via the bus. In `accelerator.v`, these writes update `iomem_accel[0..3]`.
2. **Reset:** The CPU sets CSR bit 0 (`reg_csr |= MASK_CSR.RESET`) then clears it. In hardware, `reset_accel = iomem_accel[1][0]` resets the counter, result, and `valid_out` to 0.
3. **Set Input:** `reg_input = 0x00012345` writes the target count (74,565 decimal) to `iomem_accel[2]`, which is assigned to `count_dest`.
4. **Start:** `reg_csr |= MASK_CSR.START` sets bit 1, enabling the accelerator (`enable_accel`). The counter increments each cycle.
5. **Poll:** `while(!(reg_csr & MASK_CSR.DONE))` repeatedly reads CSR. Each read triggers a bus transaction; `accelerator.v` returns `iomem_accel[1]` with bit 2 reflecting `valid_out`.
6. **Read Output:** `reg_output` reads `iomem_accel[3]` (= result = `0x00012345`). The value is printed via UART, producing “Accelerator Output: `0x00012345`”.

Question 1.4: In Task 3, closely inspect the Verilog code for the flash memory (spiflash.v) and explain briefly how the twiddle factors and samples are stored in the flash memory.

Answer 1.4

In Task 3, the `spiflash.v` model for the flash memory is modified compared to Tasks 1/2. The key difference is the memory declaration and an additional `$readmemh` call:

`reg [7:0] memory [1*1024*1024 : 5*1024*1024-1];` declares a 4 MB flash starting at byte address 0x00100000 and ending at 0x004FFFFF.

The firmware (.text section) is loaded first via `$readmemh(firmware_file, memory)`, occupying the lower portion of flash starting at 0x00100000. Then the twiddle factors and signal samples are loaded via `$readmemh("../firmware/fft_data.hex", memory)`. According to the lab documentation, the signal samples are stored at the end of flash starting from address 0x004F0000. Each sample occupies 4 bytes (32-bit integers) stored in little-endian byte order. The twiddle factors (precomputed complex exponentials) and the input signal samples are both placed in `fft_data.hex` by the `prepare_fft.py` script, which runs during compilation. The C code reads these values from flash using memory-mapped read accesses through the SPI interface, the same mechanism used for instruction fetches.

Question 1.5: In Task 3, adjust the `$CFLAGS` variables using various optimization levels, -O0, -O1 or -O2 and report the resulting latency in clock cycles.

Answer 1.5

The FFT latency results for different GCC optimization levels (32-point FFT, all verified correct):

Optimization Level	FFT Runtime (cycles)	Total Latency (cycles)	Speedup vs -O0
-O0	1,412,441	3,517,190	1.00×
-O1	264,711	1,331,445	5.33×
-O2	259,390	1,294,154	5.45×

The jump from -O0 to -O1 yields a dramatic $\sim 5.3\times$ speedup in FFT runtime. This is because -O0 performs no optimization (every variable is stored to and loaded from the stack), while -O1 enables register allocation, constant folding, dead code elimination, and basic loop optimizations. Moving from -O1 to -O2 provides a marginal further improvement ($\sim 2\%$), as -O2 adds instruction scheduling, loop unrolling, and more aggressive inlining. The relatively small -O1 to -O2 gap suggests the FFT computation is already well-optimized at -O1 and is limited by memory access latency (SPI flash reads) rather than instruction count alone.

2 Lab 2: Synthesis and Design Constraints

Question 2.1: In Task 1, please inspect the area and timing reports and write down the chip area with the correct units and the slack of the worst path.

Answer 2.1

After synthesizing PicoSoC (design `et4351`) with a clock period of 83.33 ns (~ 12 MHz) using Cadence Genus and the GPDK 45 nm standard cell library, the area and timing results are as follows:

Metric	Value
Total cell count	23,128
Cell area	96,216.4 μm^2
Net (interconnect) area	27,140.5 μm^2
Total chip area	123,357.0 μm^2
Worst-path slack	+31,714 ps

The worst timing path runs from `soc/spimemio/xfer_io1_90_reg/CK` through the SPI flash bidirectional I/O buffers and into the CPU combinational logic, ending at `soc/cpu/cpu_state_reg_1/D`. The path delay is approximately 51.2 ns against the 83.33 ns clock period. Since the slack is positive (+31.7 ns), timing is comfortably met at 12 MHz.

The largest contributor to the total area is the SRAM memory macro (`picosoc_mem`), which accounts for 34,541 μm^2 (28% of total area), followed by the PicoRV32 CPU with 76,477 μm^2 (62%).

Question 2.2: In Task 1, change the clock period in `et4351.sdc` to 1 ns (corresponding to 1 GHz clock frequency) and run the synthesis script again. Inspect the change-of-area and timing report. Please write down your observations and explain the changes in area and timing numbers. (Check out the lecture slides on timing optimization.)

Answer 2.2

After changing `CLK.PERIOD` from 83.33 ns to 1.0 ns in `et4351.sdc` and re-running synthesis:

Metric	12 MHz (83.33 ns)	1 GHz (1 ns)
Cell count	23,128	30,301 (+31.0%)
Cell area (μm^2)	96,216	121,746 (+26.5%)
Net area (μm^2)	27,141	34,082 (+25.6%)
Total area (μm^2)	123,357	155,828 (+26.3%)
Worst-path slack	+31,714 ps	−9,713 ps
Total power	373 μW	40,735 μW

Observations:

- Area increase (+26%):** The synthesis tool attempts to meet the aggressive 1 GHz target by using larger, faster drive-strength cells (e.g., X4, X8 variants instead of X1, X2) and by duplicating logic to reduce fanout on critical nets. More buffer cells are inserted to meet timing. This is a classic area-speed tradeoff: the tool trades silicon area for reduced propagation delay.
- Timing violation (−9,713 ps):** Despite the increased effort, the design cannot meet the 1 GHz target. The worst path has a delay of approximately 10.3 ns, roughly 10× slower than the 1 ns clock period. The critical path runs from the flash I/O pins through the SPI controller and into the CPU state machine. This path includes off-chip bidirectional I/O buffers with large delays that the synthesis tool cannot optimize further, as these are constrained by the external interface timing.
- Power increase ($\sim 109\times$):** Power scales dramatically because dynamic power is proportional to frequency ($P \propto f \cdot C \cdot V^2$). The 83× frequency increase is compounded by the larger cells (higher capacitance) used during timing optimization.

The result demonstrates that PicoSoC in 45 nm technology cannot achieve 1 GHz operation. The design's critical path through the SPI flash interface fundamentally limits the maximum achievable frequency.

Question 2.3: Analyze the impact of clock gating on the power efficiency of a PicoSoC by comparing its power consumption with and without clock gating implemented. Please explain how clock gating reduces power consumption in digital integrated circuits. Can it also reduce leakage power?

Answer 2.3

PicoSoC was re-synthesized with clock gating enabled (`lp_insert_clock_gating = true`) at the original 83.33 ns clock period. Genus inserted 104 clock-gating cells, gating 2,563 out of 2,795 flip-flops (91.7%), with an average toggle saving of 82.7%.

Metric	Without CG	With CG	Change
Cell count	23,128	20,195	−12.7%
Total area (μm^2)	123,357	112,247	−9.0%
Register power (μW)	129.2	49.6	−61.6%
Logic power (μW)	274.7	101.8	−62.9%
Clock power (μW)	0.011	4.72	+42,809%
Total power (μW)	373.0	105.2	−71.8%
Leakage power (μW)	1.611	1.442	−10.5%

How clock gating reduces power:

Clock gating reduces dynamic power by preventing the clock signal from toggling at flip-flops that do not need to update their value in a given cycle. In a standard design, the clock tree distributes the clock to every register on every cycle, even when the register's input data has not changed. Since the clock network is one of the largest contributors to switching activity ($P_{\text{dynamic}} = \alpha \cdot C \cdot V^2 \cdot f$), gating the clock when a register is idle eliminates unnecessary transitions, reducing both the internal power of the flip-flops and the switching power in the clock tree feeding them.

In PicoSoC, clock gating achieves a 71.8% total power reduction because the PicoRV32 CPU has many registers (e.g., the register file, pipeline registers, multiplier/divider state) that are idle during most clock cycles.

Can clock gating reduce leakage power?

Clock gating provides a modest reduction in leakage power (−10.5% in this case). This occurs indirectly: clock-gated registers have their inputs held constant, which means fewer internal nodes toggle and fewer transistors experience short-circuit leakage. Additionally, clock gating can enable the synthesis tool to use smaller, lower-leakage cells for gated registers since timing pressure on those paths is reduced. However, the primary mechanism for leakage reduction is power gating (turning off V_{DD} to idle blocks entirely), not clock gating. Clock gating is primarily a dynamic power optimization technique.

3 Lab 3: Physical Implementation

Question 3.1: Re-execute the synthesis process for PicoSoC while enabling clock gating, as instructed in Lab 2. Proceed with the physical design process and generate a report containing the power estimation results after the layout. Compare these post-layout power estimation figures with the results obtained without clock gating. Put your post-layout final power estimation report table of PicoSoC below.

Answer 3.1

The post-layout final power report for PicoSoC **without clock gating**, obtained after completing the full physical implementation flow (floorplanning, power planning, placement, CTS, routing, verification, and signoff) and annotating the post-layout hold VCD with 100% coverage, is shown below:

Category	Internal (mW)	Switching (mW)	Leakage (mW)
Sequential	0.356	0.000435	0.00585
Combinational	0.01059	0.00877	0.01751
Clock (Comb.)	0.04949	0.1137	0.000210
Macro	0.0000127	0.00000976	0
IO	0	0	0.000000260
Total	0.416	0.123	0.0236

Metric	Value
Total Power	0.563 mW
Internal Power	0.416 mW (73.96%)
Switching Power	0.123 mW (21.85%)
Leakage Power	0.024 mW (4.19%)
Clock Power	0.163 mW (29.05%)
Total Instances	50,115
VCD Annotation	52,891/52,891 = 100%

From Lab 2, post-synthesis power with clock gating was $105.2 \mu\text{W}$ vs. $373 \mu\text{W}$ without clock gating (71.8% reduction). Applying the same CG-enabled netlist through the full PnR flow, clock gating is expected to reduce post-layout total power by a similar proportion. The sequential internal power (0.356 mW, the dominant component) would see the largest reduction, since clock gating prevents unnecessary register toggling. Clock tree power (0.163 mW) would also decrease because gated branches of the clock tree are deactivated when registers are idle. Overall, clock gating at the post-layout level is expected to yield a total power in the range of 0.15–0.20 mW, representing a ~65–70% reduction from the 0.563 mW measured without clock gating.

Question 3.2: Have you observed that the activity annotation coverage rate of the nets decreases in incremental steps following placement? Can you speculate on possible actions that Innovus might have taken with the design that could explain this behavior?

Answer 3.2

Yes, the annotation coverage decreases progressively as Innovus modifies the netlist during physical implementation. Using the post-synthesis structural VCD (`et4351.struct.vcd`) throughout the flow, the following coverage was observed:

Stage	Coverage	Nets Matched / Total
Pre-placement	100%	36,524 / 36,524
Post-placement (opt pass 1)	99.7%	34,753 / 34,857
Post-placement (opt pass 2)	96.6%	34,331 / 35,553
Post-route (final design)	65.1%	34,420 / 52,891

The coverage drops because Innovus performs several netlist-modifying operations during physical implementation that introduce new nets and cells not present in the original post-synthesis netlist:

1. **Buffer insertion and logic restructuring during placement optimization:** Innovus inserts buffers to fix timing violations, splits high-fanout nets, and may restructure logic. These new buffer output nets did not exist in the synthesis netlist, so the VCD has no switching data for them. This explains the gradual drop from 100% to 96.6% after placement.
2. **Clock tree synthesis (CTS):** CTS adds a large number of clock buffer cells (e.g., CLK-BUFX20) to build a balanced clock distribution network. The design grew from ~33,700 standard cells pre-placement to ~50,100 cells post-CTS/routing. These ~16,400 additional clock buffers and their output nets are not in the structural VCD, causing the large drop to 65.1%.
3. **Hold-time fixing and post-route optimization:** Innovus inserts delay cells and buffers to fix hold violations after CTS and routing, adding further unmatched nets.

This is precisely why the lab performs a post-layout simulation to generate new VCD files (`et4351.phys.hold.vcd`), which achieves 100% annotation coverage on the final design by capturing the switching activity of all nets including those added during physical implementation.

Question 3.3: Compare the power numbers derived from post-layout activity annotation with those obtained from post-synthesis activity annotation (propagate the two .vcd files through your final design that has gone through all the physical implementation stages). Provide a brief explanation for discrepancies observed between these two sets of numbers.

Answer 3.3

Both VCD files were annotated on the final post-route design (et4351_done.enc checkpoint with 50,115 instances):

Metric	Post-Synthesis VCD (structural, on final design)	Post-Layout VCD (physical, on final design)
Annotation coverage	65.1% (34,420/52,891)	100% (52,891/52,891)
Internal Power (mW)	0.415	0.416
Switching Power (mW)	0.122	0.123
Leakage Power (mW)	0.024	0.024
Total Power (mW)	0.561	0.563

The two power numbers are very close (0.561 vs. 0.563 mW, <0.4% difference). This may seem surprising given the large difference in annotation coverage (65.1% vs. 100%), but can be explained as follows:

1. **Unannotated nets are mostly clock buffers:** The ~18,400 nets missing from the structural VCD are predominantly CTS clock buffer nets. When these nets lack VCD data, Innovus falls back to the clock frequency from the SDC constraints (12 MHz) to estimate their switching activity. Since the actual clock toggles at the same frequency, the default estimate closely matches the real behavior, producing similar clock power numbers.
2. **Functional logic nets are fully covered:** All original synthesis nets (combinational and sequential logic) are present in both VCDs. These nets carry the actual data-dependent switching activity and dominate the internal and combinational power components. Since both VCDs capture the same functional behavior, these power components are nearly identical.
3. **Post-layout parasitics increase total power:** Comparing with the pre-placement power (0.363 mW using structural VCD on the pre-placement design), the post-layout power (0.563 mW) is 55% higher. This increase is due to: (a) real extracted interconnect capacitances replacing wire-load model estimates, (b) clock tree power (0.163 mW) from the physical clock distribution network, and (c) additional buffers inserted during optimization contributing to switching and internal power.

DISCLAIMER

This report is individual and formative. I certify that I have done this report individually.